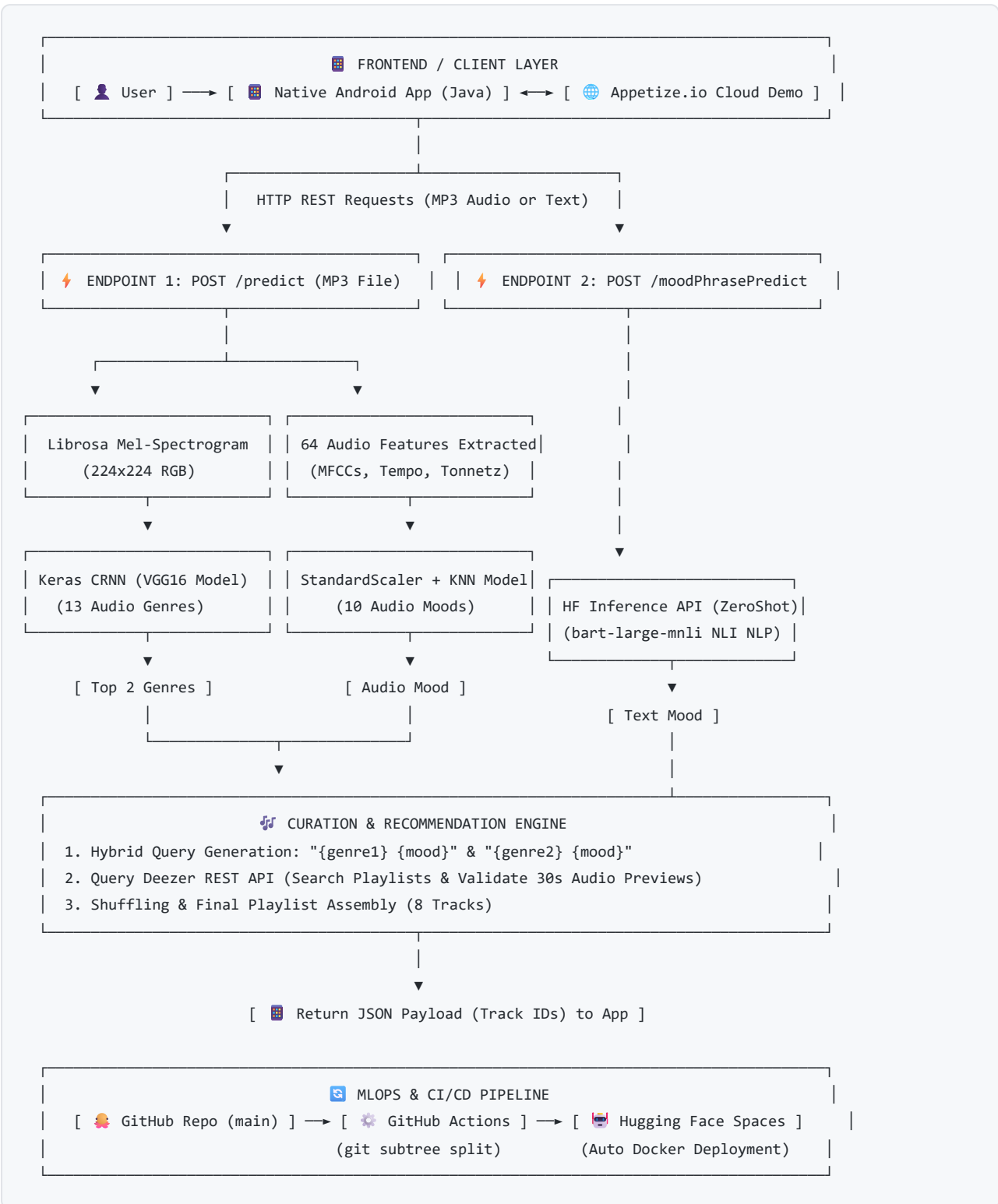


TECHNICAL CHEATSHEET & SYSTEM DESIGN: AI PLAYLIST CURATION (ARTISHOW)

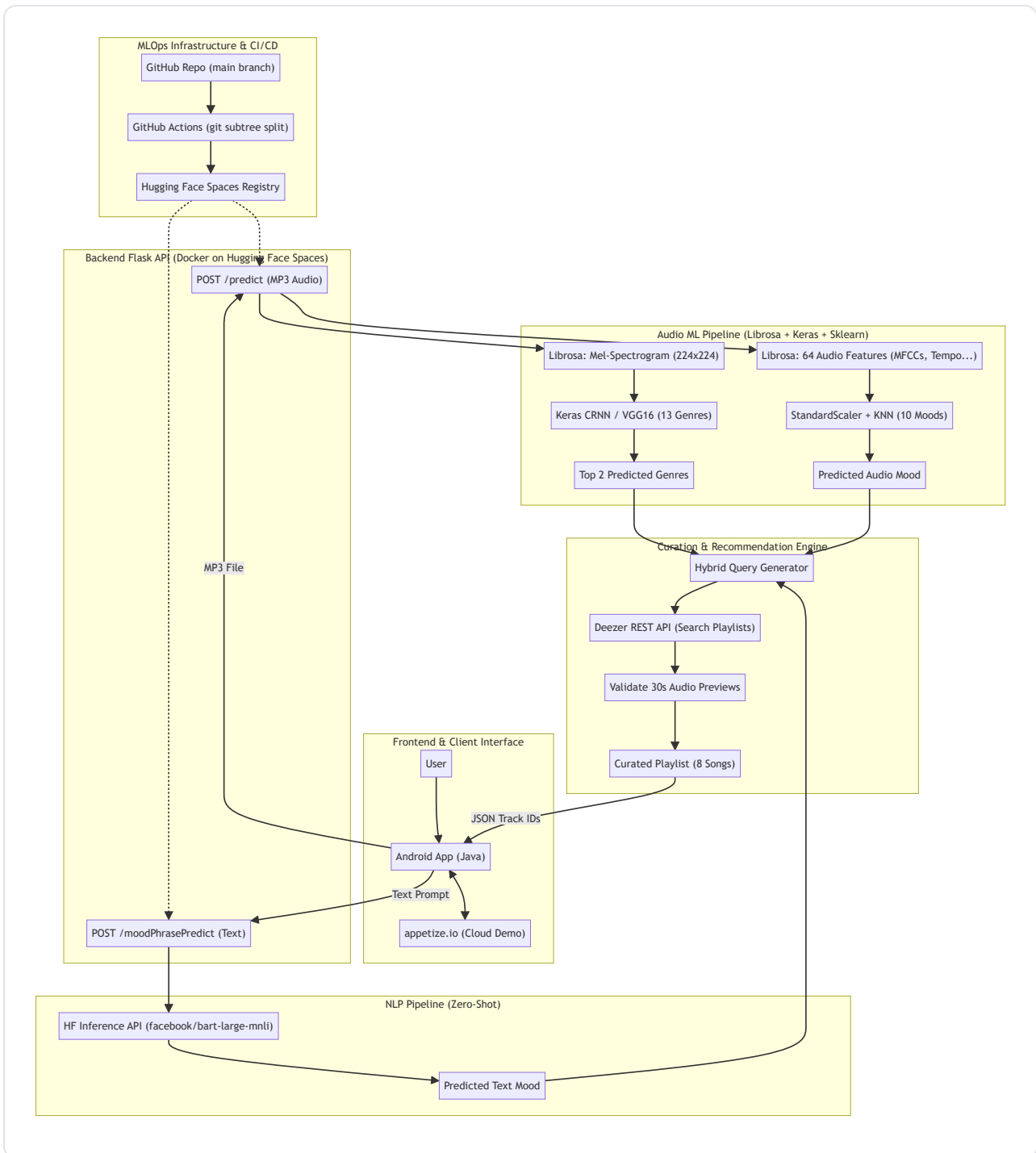
Mistral AI Interview Preparation Guide — Forward Deployed ML Engineer (FD-MLE)

1. System Design Architecture

◆ Universal Text Diagram (Universally Readable)



Mermaid Flowchart (Rendered Graph)



2. Machine Learning & Deep Learning Pipelines (Deep Dive)

A Genre Classification (Audio-to-Vision via CRNN / VGG16)

- **Core Concept:** Formulate audio signal analysis as an Audio Spectrogram Computer Vision task.
- **Audio Preprocessing (Librosa):**
- Sampling rate: `22,050 Hz`, duration fixed to `30 seconds` (wrapped/padded if shorter, truncated if longer).

- Compute **Mel-Spectrogram** using `n_mels=224 , hop_length=512` .
- Logarithmic dB transformation via `librosa.power_to_db` .
- Export as temporary RGB image **224x224 pixels** using the `magma` colormap.
- **Model Architecture:**
- Convolutional Neural Network (CRNN) based on **VGG16**, trained with **TensorFlow / Keras 3** (`modele_crnn_hd_final_13genres.keras`).
- **13 Genre Classes:** *Blues, Classical, Country, Disco, Hiphop, Jazz, Metal, Pop, Reggae, Rock, etc.*
- **Inference Strategy:** Extract **top-2 highest probability genres** (`genre1 , genre2`) via `np.argsort(predCnn[0])[-2:]` to support multi-genre hybrid tracks (e.g., *Rock/Metal*).

B Audio Mood Classification (Tabular Feature Engineering + KNN)

- **Feature Extraction (64 total dimensions):** Calculates both **mean** and **standard deviation** for each descriptor:
- **13 MFCCs** (Mel-Frequency Cepstral Coefficients) -> 26 features.
- **RMS Energy** (Root Mean Square) -> 2 features.
- **Spectral Centroid** (Brightness of audio spectrum) -> 2 features.
- **Spectral Bandwidth** -> 2 features.
- **Spectral Contrast** -> 2 features.
- **Spectral Flatness** (Noise vs. Tonal content) -> 2 features.
- **Spectral Rolloff** (Frequency below which 85% of power resides) -> 2 features.
- **Tonnetz** (Harmonic / Tonal centroid features) -> 12 features.
- **Zero-Crossing Rate** (Rate of sign changes) -> 2 features.
- **Tempo (BPM)** via `librosa.beat.beat_track` -> 1 feature.
- **Tabular Model:**
- **StandardScaler** (`scaler.pkl`) for feature normalization.
- **K-Nearest Neighbors Classifier** (`knn_model_mood.pkl`).
- **10 Mood Classes:** *dark, deep, dream, emotional, epic, happy, motivational, relaxing, romantic, sad.*

© Text Mood Classification (Zero-Shot NLP)

- **Endpoint:** `/moodPhrasePredict`
- **Model:** `facebook/bart-large-mnli` via Hugging Face Inference API.
- **Mechanism:** Zero-Shot NLI classification mapping user prompt text to the candidate mood labels.

🎵 3. Recommendation Engine & Curation Algorithm

1. Hybrid Query Generation:

- Query 1: `"{genre1} {mood}"` → Fetches 4 tracks.
- Query 2: `"{genre2} {mood}"` → Fetches 4 tracks.

4. *Fallback*: If second genre is identical or missing, falls back to `"{mood} vibe"`.
5. **Deezer REST API** (`https://api.deezer.com/search/playlist`):
6. Searches 15 playlists, filters those with > 5 tracks, randomly picks one playlist to **guarantee fresh recommendations on every run**.
7. **Validation & Assembly**:
8. Filters tracks for valid 30-second audio previews (`t['preview']`).
9. Shuffles (`random.shuffle`) the final 8 tracks to prevent block-grouping.

4. Infrastructure, MLOps & CI/CD

- **Docker Containerization**:
- Base Image: `python:3.12-slim`.
- C/C++ System Dependencies: `libsndfile1` (soundfile decoding) and `ffmpeg` (MP3 processing).
- CPU Optimization: `tensorflow-cpu` package for slim image footprints.
- **CI/CD Pipeline (GitHub Actions)**:
- Workflow file: `.github/workflows/sync_huggingface.yml`.
- Command: `git push space $(git subtree split --prefix artishow-api main):main --force`.
- Automatically deploys only the `artishow-api` subfolder to Hugging Face Spaces on push to `main`.
- **Client Interface & Testing**:
- Native Android Java Mobile Application.
- Cloud Sandboxing on **Appetize.io** for instant zero-install browser testing by juries and users.

5. Technical Interview FAQ (Mistral AI FD-MLE Focus)

Q1: "Why convert audio to 2D PNG spectrogram images instead of feeding raw tensors?"

FD-MLE Answer:

"It was a pragmatic trade-off between transfer learning and development speed. By converting the Mel-spectrogram into a 2D image (224x224), we leveraged pre-trained vision backbones (VGG16). For production scalability, we would eliminate disk I/O entirely by passing 2D float32 spectrogram tensors in-memory."

Q2: "How would you optimize inference latency and scale to millions of users?"

FD-MLE Answer:

"1) Keep audio buffers in RAM (`io.BytesIO`). 2) Convert models to ONNX Runtime / TensorRT CPU

for 3-5x faster inference. 3) Decouple long-running audio processing using an asynchronous task queue (Celery + Redis) with WebSockets/Push notifications to the Android client."

Q3: "How did your Project Manager role add concrete technical value?"

FD-MLE Answer:

"I defined the REST API contracts (error codes, payloads, fallback strategies) and set up the automated GitHub Actions CI/CD pipeline. This allowed the ML researchers and the Android developer to iterate independently without integration bottlenecks."

6. Behavioral STAR Framework (Mistral AI Angle)

Proud Technical Project

- **Situation:** 1st year Télécom Paris project. Aimed to build a complete product rather than stopping at a Jupyter notebook.
- **Task:** Project Manager & Lead Architect responsible for delivering an end-to-end ML backend and Android client.
- **Action:** Designed the multi-modal architecture (CNN + KNN + Zero-Shot NLP), containerized with Docker, built CI/CD to Hugging Face Spaces, conducted user testing.
- **Result:** **Top 5 Jury Favorite** at Télécom Paris Showcase; highly rated by test users.

Self-Learning & Curiosity

- **Situation:** Started the project in 1st year before taking any formal Machine Learning courses.
- **Task:** Autonomously master audio signal processing and deep learning architectures on a tight timeline.
- **Action:** Studied research papers, Librosa documentation, and online courses; prototyped and evaluated feature extraction and model architectures iteratively.
- **Result:** Successfully built a functional end-to-end ML platform and onboarded team members onto the pipeline.